

Tuning Large Oracle Apps

Mark Trevor

WR Systems Ltd



So, you've assembled a large staff of ORACLE Forms and Reports developers to complete the physical model development of a large scale system. The functional requirements require you to meet certain benchmarks of performance or process large amounts of data.

It's not surprising at all to find that most ORACLE developers know very little about applications level performance tuning. There is quite a bit that the developer can do to increase the performance of the code he writes.

This paper discusses in basic terms the main tools available to the ORACLE applications developer and how they can be utilized to tune applications for increased performance. It outlines steps to take when a bottleneck is found and how these tools can help identify potential problems before they eat up valuable development resources. The main intent of this paper is not to provide an exhaustive description of each of these tools, but rather to provide a practical step by step approach to applications performance tuning which can be easily adopted by the average ORACLE applications developer with as little distraction from his or her main objective (developing the software) as possible. This paper explains, in general terms, how the ORACLE Optimizer works; it explains how tools such as Explain Plan and ORACLE Hints can be used to manipulate the Optimizer; it provides guidelines to follow when setting up indexes; and it identifies and explains the different approaches to query optimization.

The Optimizer

Any attempt to solve performance problems within your applications must start with an understanding of the ORACLE Optimizer and its role in increasing performance. When a SQL SELECT, UPDATE, INSERT or DELETE statement is processed, ORACLE must access the data referenced by the statement. The Optimizer portion of ORACLE is used to determine the most efficient path to reference the data. The optimizer formulates execution plans and attempts to choose the best plan for executing a statement.

In ORACLE version 7.xx and up, the optimizer uses one of two techniques to formulate execution plans for SQL statements: a cost-based approach or a rule-based approach. The cost-based approach generally chooses an execution plan which is as good as, or better than, the plans chosen by the rule-based approach, especially for large queries with multiple joins or multiple indexes. The cost-based approach also improves productivity by eliminating the need for you to manually tune your SQL statements yourself; meaning that you need not be concerned with the order of the tables in the WHERE clause or turning on or off indexes by modifying the columns in the WHERE clause. Why then have a rule-based approach? The rule-based approach exists mainly to accommodate code written in versions of ORACLE earlier than version 7. In addition, problems with the cost-based optimizer in version 7.1 and 7.2 have made it necessary to use the rule-based approach as a work-around in some cases. It is important though to keep in mind that the rule-based approach will be eliminated in future versions of ORACLE.

If your version of ORACLE supports cost-based optimization and you have no specific reason to use the rule-based approach, I recommend that you do not use it. However, the way the rule-based optimizer works is as follows. The optimizer uses a set of rules to determine the best execution plan for a SQL statement. The execution plan is derived by the way the tables are arranged in the WHERE clause, the availability of indexes to be used along with a few general rules applied across the board. These rules will cause the optimizer to choose the same query execution plan regardless of the underlying data distribution. For example, if the optimizer finds an index on a table, it will use it even if using the index will not eliminate a majority of the rows, thus slowing the performance of the statement. When using the rule-based approach, the developer of the query would need to explicitly turn off the index by modifying one of its columns in the WHERE clause of the statement (or by some other means), or suffer slower than necessary performance. And if the distribution of the underlying data in the tables changed such that the index was needed, the developer would have to go back and change the statement accordingly.

No matter which optimization approach is used, the optimizer's objective is the same; to determine the best path for accessing the data. This is very important because the performance implications are virtually unlimited. It may, for instance take several minutes to do a full table scan on a table with hundreds of thousands of rows. If the optimizer knows to use a unique index on the table, a single row can instead be retrieved almost instantly. The savings in retrieval time would be several minutes. That may not sound like much, but if you're the one sitting behind the keyboard waiting for a response, it could be very significant. That is however the very tip of the iceberg. Let's say this hundred thousand row table is now being joined to another table which has one hundred thousand rows in it as well. If the optimizer knows to use the unique index on this table the result is still retrieved almost instantly. If it does not know to use the index and instead chooses to do a full table scan on this table for every row in the second table, the result could take hours if not days. Add joins to a few more similar tables and without a good query optimization path, the results may never come back.

How does the cost-based Optimizer know what path to choose? The cost-based optimizer uses statistics such as the number of records in the table and the distribution and selectivity (discussed later) of the data in the tables to help it determine the optimal query execution plan. Statistics must therefore be generated for all tables and indexes accessed by your SQL statements before using the cost-based approach (generating statistics for tables automatically gathers statistics for the indexes on that table). If the size and data distribution of these tables changes frequently, you should generate these statistics on a regular basis to ensure they accurately represent the data in the tables. ORACLE can generate statistics using the following techniques.

- * Estimation based on random data sampling.
- * Exact computation.

Since collecting statistics on a table prevents SELECT, INSERT, UPDATE, and DELETE statements from accessing the table during the time the statistics are being calculated, estimation (which is much faster) is especially useful. It minimizes the

time the tables are unavailable. Exact computation should be used when estimation produces skewed results.

The following example statement generates statistics for the PERSON table and its indexes:

```
ANALYZE TABLE PERSON ESTIMATE STATISTICS SAMPLE 20 PERCENT;
```

The ORACLE Optimizer will use the cost-based approach to optimization only if statistics exist for the tables accessed by the statement and if the OPTIMIZER_GOAL session parameter is set to a cost-based value such as CHOOSE. (This is generally the default setting.) If these conditions are not met, the optimizer will revert to optimizing the statement by a set of rules which do not take the actual number of rows in the tables or the distribution of the data into consideration (rule-based approach).

You can also enable cost-based optimization at the session and statement level. To enable cost-based optimization for your session only, issue the ALTER SESSION statement with the OPTIMIZER_GOAL parameter value of CHOOSE, ALL_ROWS or FIRST_ROWS. To enable cost-based optimization for an individual SQL statement, use the ALL_ROWS or FIRST_ROWS hint (discussed later).

So, the first step to performance tuning the application is to make sure you have enabled the cost-based approach to query optimization and verify that statistics on the target tables are up-to-date. If for some reason, the cost-based approach is not available to you, then you need to write the SELECT, UPDATE, INSERT or DELETE statement in such a way that the best optimization path is available to the rule-based optimizer (techniques for doing this are discussed later).

The next step is to choose the appropriate goal for the cost-based optimizer's execution plan. The execution plan produced by the optimizer can vary greatly depending on the optimizer's goal. The types of joins that the optimizer chooses can have a great impact on the way the results are returned. For example, optimizing for best throughput, (or the minimum time to return all rows accessed by the statement) is likely to result in a full table scan rather than an indexed scan. It would also more likely result in a sort-merge join rather than a nested loops join. Optimizing for the best response time, (or the minimum time to return the first row accessed by the statement) is more likely to result in an indexed scan or a nested loops join. This is because sort-merge join operations may return the entire query result faster, while the nested loops operation may return the first row faster. If the goal is best throughput, the optimizer is more likely to choose the sort-merge join. If the goal is best response time, the optimizer is more likely to choose a nested-loops join. It is therefore desirable to choose a goal for the optimizer based on the needs of the application. In general, if the statement is to return rows to the screen, response time would be most important because the user can proceed once the first row has been returned to the screen. However, if the statement returns rows to a report, throughput would be generally be more important because the report would be unavailable until all rows were retrieved anyway. By default, the cost-based approach optimizes for best throughput.

The goal of the cost-based approach can be set at the session or statement level. To change the goal of the cost-based approach for all SQL statements in your session,

issue the ALTER SESSION statement with the OPTIMIZER_GOAL parameter. To specify the goal of the cost-based approach for an individual SQL statement, use the ALL_ROWS or FIRST_ROWS hint (discussed later).

Indexing

It has been my experience that the solution to the vast majority of performance problems boils down to creating indexes or making existing indexes visible or accessible to the optimizer. Without a doubt, a few well placed indexes can produce the greatest improvement in application performance, especially when the volume of data is high. More than a few applications programmers have spent hours and hours reengineering database structure or their code, or messing with system parameters in an attempt to improve application performance only to find out that the problem was that the optimizer was not using the indexes which they thought they were. It is important to understand when to use indexes and when not to use indexes.

Indexes should generally be used when the statement will result in a small percentage of the total number of rows in the table being returned. In this case, the statement is said to have good selectivity. For example, if I am selecting all of the employees in the EMPLOYEE table with a particular social security number or name, since the number of rows in the table with a particular social security number or name is expected to be low, processing the statement using an index on the social security or name field would undoubtedly improve performance. If, on the other hand, I was selecting all of the EMPLOYEES who are female, I would probably not want to use an index since the number of rows which meet the criteria would be pretty high (probably around 50 percent). It is important to understand that processing a statement through an index takes overhead. The more records processed through the index, the more overhead incurred. The point at which the overhead outweighs the benefit of the index varies, but is generally accepted to be between 15 and 25 percent. That is, if the statement retrieves more than 15 to 25 percent of the rows in the table, it will probably be cheaper to do a full table scan rather than using the index. In this case, the statement is said to be not very selective (bad selectivity).

The next step in the process of resolving your performance problem should then be to make sure that indexes are placed on the appropriate tables and that the statement is actually using the indexes. The ORACLE optimizer will decide whether or not to use an index which is available to it, but your job is to make sure that the index is indeed available to the optimizer for the given statement. In addition to creating the indexes and making them visible to the optimizer, you must also make sure that you don't inadvertently make them unavailable to the optimizer. For example, the optimizer cannot use the index if the columns in the index are modified in the WHERE clause. Remember, to avoid the following when using indexes, because they can make indexes unavailable to the optimizer.

- * Avoid modifying the indexed columns in the WHERE clause.
- * Avoid using NOT, !=, <> or || in the WHERE clause.
- * Avoid using calculations on indexed columns in the WHERE clause.
- * Avoid using columns which contain NULLs in your indexes.

In addition, do the following whenever possible.

- * Remember to use the leading portion of composite indexes (e.g. if your index is made of more than one column, and you are only using part of the index, make sure the part you use is in the leading portion of the index).
- * Use UNION instead of OR.
- * Remember to consider internal conversions. (For example, if a numeric column is compared to an alphanumeric column, the alphanumeric column will automatically be converted to numeric by ORACLE, thus preventing the optimizer from being able to use the column in an index.)

Explain Plan Tool

Once you have verified that indexes are in place and that to the best of your knowledge, they are available to the optimizer to use, the next step is to see what the optimizer is really doing. This can be done by using the Explain Plan tool. The Explain Plan tool will allow you to see the exact execution plan which the optimizer will choose when executing your statement. This lets you see if the indexes you expect to be used are indeed being used. It can also give you some idea of how long the statement will take to execute.

The sample file below contains the statements necessary to use the Explain Plan to evaluate ORACLE statements. To use it, replace USER_NAME 'with something unique which identifies you. Next, paste the SQL statement which you are evaluating in the file where indicated. Then execute the file from the SQL prompt and observe the output.

EXPLTEST.sql

delete from plan_table

where statement_id = 'USER_NAME'

/

explain plan set statement_id = 'USER_NAME'

into plan_table for

(Place your SQL statement here)

/

select lpad(' ',1*level)||operation||'('||options||')'||object_name||' '||object_type
results

from plan_table

```
connect by prior id = parent_id

and statement_id = 'USER_NAME'

start with id = 1

and statement_id = 'USER_NAME'

/
```

Below is a sample SQL statement which selects backorder quantities for stock in inventory.

```
SELECT I.STOCK_NBR, B.BACKORD_QTY

FROM INVENTORY I, BACKORDER B

WHERE I.INVENTORY_ID = B.INVENTORY_ID

AND I.INVENTORY_ID = 123

/
```

Consider the following output on the above SQL statement which came from the Explain Plan tool. It tells me that a nested loops join is being done to link the two tables. It tells me that a full table scan is being done on the INVENTORY table and the records returned from that table are being joined via a range scan to the BACKORDER table's BACKORDERS_FOR_FRGN index. The BACKORDERS_FOR_FRGN index then links by ROWID to the actual BACKORDER table (this is necessary because the column being returned from the BACKORDER table is not in the index).

```
NESTED LOOPS()

TABLE ACCESS(FULL)INVENTORY

TABLE ACCESS(BY ROWID)BACKORDER

INDEX(RANGE SCAN)BACKORDERS_FOR_FRGN NON-UNIQUE
```

It would be a good idea to spend some time learning to understand the output from the Explain Plan tool. The value in doing this obviously increases the more bottlenecks you have to fix, however, even if you are not sure at first how much you will need it, I still strongly recommend that you take the time. Since explaining all of the ins and outs of the Explain Plan tool's output could easily be the subject of an entire paper, and since it was not my intent to get to that level, I will just touch on a few of the basics.

In general, full table scans are your most glaring place for improvement. Replacing them with indexed searches will give you the most bang for the buck! Any time you

see that the table access is by a full table scan, it should stick out as a possible place for improvement.

In the example statement above, it shows us that no index is being used on the INVENTORY table. There is clearly room for improvement here because we are giving it a distinct INVENTORY identifier. In this case a quick check might reveal that no index existed on the INVENTORY_ID field of the INVENTORY table. Adding the index would change the Explain Plan output as follows.

NESTED LOOPS()

INDEX(UNIQUE SCAN) INVENTORY_PK UNIQUE

TABLE ACCESS(BY ROWID) BACKORDER

INDEX(RANGE SCAN) BACKORDERS_FOR_FRGN NON-UNIQUE

Note that the FULL access has been replaced by a UNIQUE SCAN. This output is much better and now, the statement will return the results instantly rather than taking several minutes.

The type of join operation used by the optimizer can significantly impact performance. The ORACLE server uses the following three basic types of join operations.

- * Nested loops returns the first records to the next operation quickly. It is the most common way ORACLE server performs joins, and in most cases, it indicates that an index is available for use during the join.
- * Merge Joins does not return records to the next operation until all of the rows have been processed. It is usually used when indexes are either unavailable or disabled by the statement's syntax.
- * Hash Join is similar to nested loops in that it loops through rows coming from each step of the plan, however instead of going to an index or table to retrieve the row, it builds an internal cache structure in memory from which to work.

Understanding the distribution of data within the tables which are being joined and the amount of data (e.g. number of rows) in each table is also important. As an example, the nested loops join is a directional operation: if you join two tables together, you will get different performance depending on which table the optimizer chooses as the driving table. In the example above, the optimizer choose the INVENTORY table as the driving table and joined each row from that table their corresponding rows in the BACKORDER table. This was desirable since we expect only one row with INVENTORY_ID = 123. Thus, only one pass through the BACKORDER table's index is necessary. The optimizer will try to choose the table which returns the fewest rows as the driving table. The cost-based optimizer determines this by the available statistics on the tables (among other things). The

rule-based optimizer determines this by the placement of the table in the FROM clause (among other things).

Oracle Hints

At times you may know more about the distribution of the data, indexes or other factors which the optimizer uses to determine the best execution plan than ORACLE does. You may want to tell the optimizer to use or not to use a particular index. Or, it may be helpful to have the optimizer use the rule-based approach to optimization even though your default approach is cost-based. Through the use of ORACLE hints, the optimizer can be instructed to do these kinds of things. ORACLE hints can be applied to simple select, update or delete statements, to the parent or subquery of a complex statement or even to a part of a compound query, in order to specify one or more of the following.

- * The optimization approach for a SQL statement.
- * The goal of cost based optimization (e.g. best throughput or best response time).
- * The join order for a SQL statement.
- * The join operation for a statement (e.g. sort merge or nested loops).

In short, ORACLE hints can be used to give you control over the choices that the optimizer makes when establishing the execution plan for your SQL statement. The syntax for using hints is as follows:

```
SYNTAX : select | delete | update /*+ hint */
```

```
from ...;
```

```
select | delete | update --+ hint
```

```
from ...;
```

As you can see, the hint is buried inside a comment on the select line of the WHERE clause. This is the only place in the statement where hints can be used. Since it is buried inside a comment, it is important to understand that if the syntax is incorrect or the hint is inappropriate, it will be treated as a comment and ignored. This means that particular attention should be paid to detail when using hints.

Consider the same query we used in the previous example. If for some reason you knew that a full table scan on the BACKORDER table would be faster than using the index, you could use an ORACLE hint to tell the optimizer to ignore the index on the BACKORDER table and instead do a full table scan. This might happen if, for instance, you knew that the vast majority of the records currently in the BACKORDER table were related to the INVENTORY record with INVENTORY_ID = 123. The new SQL statement with the ORACLE hint would look like this.

```
SELECT /*+FULL(B) */ I.STOCK_NBR, B.BACKORD_QTY
```

```
FROM INVENTORY I, BACKORDER B

WHERE I.INVENTORY_ID = B.INVENTORY_ID

AND I.INVENTORY_ID = 123

/
```

Note that the hint references the BACKORDER table by its alias rather than by the table name itself. Beware, this is a necessary thing, and is a matter of syntax, but it can cause confusion. The new Explain Plan output for this statement is as follows.

```
NESTED LOOPS()

INDEX(UNIQUE SCAN)INVENTORY_PK UNIQUE

TABLE ACCESS(FULL)BACKORDER
```

As you can see, the optimizer took the hint and did a full table scan on the BACKORDER table rather than using the available index. Hints such as USE_NL and USE_MERGE can be used to choose the join type. Hints such as FULL, ROWID and INDEX can be used to choose the access methods for the join.

The ORDERED hint will cause the tables to be joined in the order they are written in the from clause, from left to right (useful when you need to change the driving table). In addition, the ALL_ROWS, FIRST_ROWS and RULE hints can be used to choose the optimization goal and approach for the SQL statement. Again, the intent was not to discuss these in detail, but rather to provide some exposure to them and show how they can be used to effect performance.

Conclusion

The join operation is one of the most important aspects of the relational database and it has a profound impact on performance and specifically response-time. Understanding the way the ORACLE optimizer determines execution plans to accomplish the join, and understanding the tools which allow you to manipulate the optimizer in its choice of these plans, will allow you to more easily increase productivity and the overall user satisfaction of your system. To that end, all of the tools discussed in this paper can and should be employed by your developers to help solve performance problems in their applications. If developers use these techniques, in the order in which they are presented, the resolution to the vast majority of performance problems encountered will become quickly clear. It is my sincere hope that this paper's introduction to the tuning concepts and tools will act as a roadmap to the art of solving performance problems and that the steps presented here will serve as a starting point for developers who are beginning to explore the vastly misunderstood world of performance tuning large applications.